

# MUFG — Full Stack Analyst Intern

## Comprehensive Test & Interview Prep Guide

Built from the official Job Description + reported MUFG/MGS interview & assessment patterns (Glassdoor, Indeed, candidate reports). Covers Reasoning Ability, Technical Ability, and HR/Behavioral rounds.

### What MUFG's process actually looks like (from real candidate reports)

- Process usually runs: **Online Assessment** (aptitude/reasoning, sometimes an attention-to-detail or Excel section) → **Technical/HR phone screen** → 1-2 **Technical interviews** (live coding on notepad/shared screen — OOP, DSA, SQL, DBMS) → **Final round** with AVP/VP/Manager (behavioral + culture fit, sometimes a short presentation).
- Reported live-coding themes: OOP concepts written as code (abstraction, overriding, sharing properties), basic DSA problems, DBMS/normalization questions, and writing SQL queries directly on a notepad.
- Reported systems/infra questions for tech roles: multithreading, distributed systems, Docker, basic Kafka awareness, AWS services, Git, and CI/CD.
- Reported conceptual ask: 'Explain what Big Data is and which companies use it and how' — be ready for one broad conceptual explainer question like this.
- Behavioral is weighted heavily even in technical rounds: 'Tell me about yourself', 'Strengths & weaknesses', 'Why MUFG', 'Walk me through your resume', 'Tell me about a challenge you faced in a project'.
- Interview difficulty is generally rated **moderate (not brutal)** — candidates consistently describe it as conversational rather than adversarial, so calm, clear explanations matter more than perfect answers.

*Since this is a fresh JD-based intern/analyst hire, expect a shorter online assessment (reasoning + basic technical MCQs / short code) rather than a 90-minute DSA marathon. Prioritize breadth and clear verbal explanation over deep leetcode-hard prep.*

# Contents

---

1. Reasoning Ability — Quantitative Aptitude
2. Reasoning Ability — Logical Reasoning
3. Reasoning Ability — Verbal Ability
4. Core Java & OOP
5. J2EE, Spring & REST APIs
6. JavaScript Fundamentals
7. React.js (Hooks, Redux/Flux)
8. HTML + CSS
9. SQL & Databases
10. Data Structures & Algorithms
11. System Design / Distributed Systems Basics
12. Git
13. Application Security (OWASP)
14. Agile / Scrum / Waterfall
15. AWS, Docker & CI/CD Basics
16. LLMs / GenAI Basics
17. HR & Behavioral Round
18. Test-Day Strategy & Priority Checklist

# 1. Reasoning Ability — Quantitative Aptitude

---

Standard bank/IT-services online assessment topics. Speed matters more than depth — aim for <60-90 sec per question.

## Key topics & formulas

- **Percentages:** % change =  $(\text{New}-\text{Old})/\text{Old} \times 100$ . Successive % change: A% then B%  $\rightarrow$  net =  $A+B+(AB/100)$ .
- **Profit & Loss:** Profit% =  $(\text{SP}-\text{CP})/\text{CP} \times 100$ . Marked price/discount:  $\text{SP} = \text{MP} \times (1 - \text{discount}\%)$ .
- **Ratio & Proportion:** If  $a:b = x:y$ , combine via LCM of b and x for direct comparison.
- **Time, Speed & Distance:** Speed = Distance/Time. Relative speed: same direction = difference, opposite = sum. km/h  $\rightarrow$  m/s: multiply by 5/18.
- **Time & Work:** If A does a job in 'a' days, A's 1-day work =  $1/a$ . Combined work = sum of individual rates.
- **Simple & Compound Interest:** SI =  $P \times R \times T/100$ . CI =  $P(1+R/100)^T - P$ .
- **Averages:** Average = Sum/Count. Watch for 'replaced/added' average-shift questions.
- **Permutations & Combinations, Probability:**  $nPr = n!/(n-r)!$ ,  $nCr = n!/(r!(n-r)!)$ . Probability = favorable/total outcomes.
- **Number Series & Number System:** LCM/HCF, divisibility rules, finding next term in a sequence.
- **Data Interpretation:** Reading bar/line/pie charts and tables quickly, computing ratios/percentages from given data.

**Q: A shopkeeper marks an item 40% above cost price and gives a 25% discount. What is his net profit/loss %?**

$\text{MP} = 1.4 \times \text{CP}$ .  $\text{SP} = 0.75 \times 1.4 \times \text{CP} = 1.05 \times \text{CP} \rightarrow 5\% \text{ profit}$ .

**Q: If the speed of a train is 90 km/h, what is it in m/s?**

$90 \times 5/18 = 25 \text{ m/s}$ .

**Q: A can finish a task in 10 days, B in 15 days. Working together, how long will they take?**

Combined rate =  $1/10 + 1/15 = 1/6 \rightarrow 6 \text{ days}$ .

**Q: Practice set to run through tonight:**

- 2-3 percentage/profit-loss word problems
- 2 time-speed-distance (including relative speed: trains crossing)
- 2 time-and-work problems (including 'pipes filling/draining a tank' variant)
- 1-2 simple/compound interest problems
- 1 data interpretation table/chart with 3-4 sub-questions

## 2. Reasoning Ability — Logical Reasoning

---

### Key topics

- **Coding-Decoding:** letter/number shift patterns, symbol substitution.
- **Blood Relations:** drawing a quick family tree from statements.
- **Syllogisms:** 'All/Some/No' statements → valid conclusions using Venn diagrams.
- **Seating Arrangement / Puzzles:** linear and circular arrangements with conditional clues.
- **Series Completion:** number, letter, and figure series — find the pattern (arithmetic, geometric, alternating).
- **Direction Sense:** tracking movement (N/S/E/W turns) to find final position/distance.
- **Statements & Assumptions / Arguments:** distinguishing a valid logical assumption from an unstated opinion.
- **Data Sufficiency:** deciding if given statement(s) are enough to answer a question, without solving it fully.
- **Venn Diagrams / Set-based logic:** overlapping category questions.

#### Q: Tip for syllogism questions

Always draw the Venn diagram rather than reasoning purely in your head — 'Some A are B' does NOT imply 'Some B are A' is automatically false, and these traps are the most common wrong-answer source.

#### Q: Tip for puzzles/seating arrangement

Read all clues once fully before drawing anything, place the most restrictive/absolute clue first (e.g., 'X sits at one end'), then fill relative clues around it.

### 3. Reasoning Ability — Verbal Ability

---

- **Reading Comprehension:** 1-2 short passages with inference-based questions — read the questions first, then skim for answers.
- **Sentence Correction / Grammar:** subject-verb agreement, tenses, articles, prepositions.
- **Synonyms / Antonyms:** common corporate/business vocabulary.
- **Sentence Completion / Fill in the Blanks:** context-based single or double blanks.
- **Para Jumbles:** rearranging 4-5 jumbled sentences into a logical paragraph — find the opening sentence (general statement) and closing sentence (conclusion) first.
- **Error Spotting:** identifying the segment of a sentence with a grammatical error.

*This section is usually the fastest to clear if English is already comfortable for you — don't over-invest prep time here relative to quant/logical/technical.*

## 4. Core Java & OOP

---

### Must-know theory

- Four pillars: **Encapsulation, Abstraction, Inheritance, Polymorphism** — have a one-line definition + code example ready for each.
- **Overloading** (compile-time, same method name/different params) vs **Overriding** (runtime, subclass redefines superclass method, needs same signature).
- **Abstract class vs Interface:** abstract class can hold state/constructors; interface (Java 8+) allows default/static methods but no instance state. Interface = 'can-do' contract, abstract class = 'is-a' shared implementation.
- **final vs finally vs finalize()** — three unrelated keywords, a classic trick question.
- **Static vs instance** members/methods; static binding vs dynamic (runtime polymorphism) binding.
- **== vs .equals()** — reference equality vs value equality; why the String pool matters.
- **hashCode()/equals() contract:** equal objects must have equal hash codes; breaking this corrupts HashMap/HashSet behavior.
- **Exception hierarchy:** Throwable → Error / Exception → checked vs unchecked (RuntimeException and its subclasses).
- **Collections internals:** ArrayList (dynamic array, O(1) access) vs LinkedList (doubly-linked, O(1) insert/delete at ends); HashMap vs TreeMap (sorted, Red-Black tree) vs LinkedHashMap (insertion order).
- **HashMap internals:** buckets + hashing, load factor (default 0.75), resizing/rehashing, collision handling — linked list per bucket, converts to a tree when a bucket exceeds 8 entries (Java 8+).
- **Multithreading:** Thread vs Runnable, synchronized keyword, volatile, deadlock conditions, wait()/notify(), ExecutorService thread pools (reported as an actual MUFG interview topic).
- **Generics:** type erasure, bounded types like <T extends Comparable<T>>.
- **Java 8+:** Streams API (map/filter/reduce/collect), lambda expressions, Optional, functional interfaces (Predicate, Function, Supplier, Consumer).
- **Garbage collection:** generational GC basics, when an object becomes eligible for collection.
- **String immutability:** why Strings are immutable, and StringBuilder vs StringBuffer (thread-safety difference).

### Likely questions (some reported directly from MUFG interviews)

- Write code demonstrating abstraction and method overriding live (reported live-coding exercise).
- Why is String immutable in Java?
- Difference between JDK, JRE, and JVM.
- What happens if you override equals() but not hashCode()?
- How does HashMap handle collisions internally?
- Explain how multithreading works in Java — synchronized vs volatile, and how you'd avoid a deadlock.
- Diamond problem — how do Java interfaces avoid it?
- Trace/debug a small recursive function by hand.

## 5. J2EE, Spring & REST APIs

---

- **Servlet lifecycle:** `init()` → `service()` → `destroy()`.
- **JSP vs Servlet:** JSP compiles down to a servlet; separates view from logic.
- **Session management:** cookies, `HttpSession`, URL rewriting.
- **MVC pattern** in a J2EE/Spring context.
- **Spring / Spring Boot basics:** `@RestController`, `@Service`, `@Repository`, `@Autowired`, Dependency Injection, IoC container — Spring Boot is the realistic modern stand-in for 'J2EE' in most current interviews.
- **REST principles:** statelessness, proper HTTP verbs (GET/POST/PUT/PATCH/DELETE), idempotency, correct status codes (200/201/400/401/403/404/500).
- **JDBC:** `Connection`, `PreparedStatement` (prevents SQL injection) vs `Statement`, `ResultSet`.

### Q: What is dependency injection and why does it matter?

It's a design pattern where an object's dependencies are provided externally (usually by a framework/container) rather than the object creating them itself — this decouples components, making code more testable and swappable.

### Q: Difference between PUT and PATCH?

PUT replaces the entire resource; PATCH applies a partial update to it.

### Q: Why use PreparedStatement over Statement?

`PreparedStatement` pre-compiles the SQL with parameter placeholders, so user input is treated strictly as data — not executable SQL — preventing SQL injection.

## 6. JavaScript Fundamentals

---

*Explicitly named in the JD as 'JavaScript Fundamentals' — expect direct, focused questions here.*

- **var vs let vs const:** var is function-scoped and hoisted as undefined; let/const are block-scoped with a temporal dead zone. Classic trap: a for-loop with var vs let inside setTimeout.
- **Hoisting:** function declarations are fully hoisted; var is hoisted as undefined; let/const are hoisted but stay uninitialized until their line runs.
- **Array methods:** map/filter/reduce/forEach — know the difference (returns new array vs single value vs nothing); mutating (push, splice) vs non-mutating (slice, concat, map).
- **DOM manipulation:** querySelector vs getElementById, event bubbling vs capturing, event delegation, addEventListener.
- **Prototypal inheritance:** every object has an internal [[Prototype]] link; Object.create(); prototype chain lookup; ES6 'class' is syntactic sugar over this, not classical inheritance.
- **Object creation patterns:** constructor functions, Object.create, factory functions, ES6 classes.
- **Equality:** == (with type coercion) vs === (strict, no coercion).
- **this keyword:** depends on the call-site — global, method call, arrow function (lexical this), and explicit binding via call/apply/bind.
- **Closures:** an inner function retains access to its outer function's scope even after the outer function returns — used for counters, memoization, data privacy.
- **Async JS:** Promises, async/await, then/catch, the event loop, microtask vs macrotask queue ordering, Promise.all vs Promise.race.
- **ES6+:** destructuring, spread/rest operators, template literals, default parameters, arrow functions, ES modules (import/export).

### Q: Classic trace question

for (var i=0; i<3; i++){ setTimeout(()=>console.log(i)) } prints 3,3,3 (var is shared/function-scoped). Replacing var with let prints 0,1,2 (each iteration gets its own block-scoped binding). Be ready to explain why.

### Q: What is a closure? Give a real use case.

A closure is a function bundled with references to its surrounding lexical scope. Use case: a counter factory — function makeCounter(){ let c=0; return ()=>++c; } — 'c' stays private and persists across calls.

### Q: Explain the event loop briefly.

JS is single-threaded; synchronous code runs on the call stack. Async callbacks go into queues — microtasks (Promises) are drained before macrotasks (setTimeout, events) on each loop tick, which is why Promise callbacks run before setTimeout callbacks even with a 0ms delay.

### Q: Implement debounce (common live-coding ask).

```
function debounce(fn, delay){ let t; return (...args)=>{ clearTimeout(t); t=setTimeout(()=>fn(...args), delay); }; }
```

## 7. React.js (Hooks, Redux/Flux)

---

- **Virtual DOM & reconciliation:** diffing algorithm, why keys in lists matter for correct re-use of DOM nodes.
- **Hooks:** useState, useEffect (dependency array behavior: empty [] = mount only, no array = every render, [dep] = on dep change), useContext, useRef, useMemo vs useCallback (memoize a value vs a function reference).
- **Class lifecycle equivalents** (for comparison):  
componentDidMount/componentDidUpdate/componentWillUnmount ↔ useEffect variants.
- **Props vs State:** one-way data flow; 'lifting state up' when siblings need to share data.
- **Controlled vs uncontrolled components** (form inputs bound to state vs using refs).
- **JSX:** transpiles to React.createElement calls.
- **Redux/Flux** (explicitly named in JD): unidirectional flow — Action → Reducer (pure function) → Store → View. Middleware like redux-thunk handles async actions.
- **Context API vs Redux:** Context is fine for simple/low-frequency global state; Redux scales better for complex, frequently-updated state with devtools/middleware needs.
- **Performance:** React.memo, code-splitting via React.lazy, avoiding unnecessary re-renders.

### Q: Why do we need keys in lists? What happens without them?

Keys let React match array items across renders to reuse/update the correct DOM node instead of re-creating it. Without stable keys (or using array index on a reorderable list), React can mismatch state between items, causing bugs like inputs retaining the wrong value after reordering.

### Q: Difference between useMemo and useCallback?

useMemo memoizes a computed value; useCallback memoizes a function reference itself (so a child wrapped in React.memo doesn't re-render just because a new function instance was passed down).

### Q: Why must a Redux reducer be a pure function?

Predictability and debuggability — same input always produces same output, no side effects, which enables time-travel debugging and reliable state reproduction.

## 8. HTML + CSS

---

- **Semantic HTML:** improves accessibility and SEO (nav, main, article, section vs generic div).
- **Box model:** content-box (default) vs border-box (padding/border included in declared width).
- **Flexbox vs Grid:** Flexbox = 1D layout (row or column alignment); Grid = 2D layout (rows + columns simultaneously).
- **Positioning:** static, relative, absolute (relative to nearest positioned ancestor), fixed (viewport), sticky (hybrid).
- **Specificity:** inline > ID > class/attribute/pseudo-class > element, with !important overriding all (avoid it).
- **Responsive design:** media queries, rem/em vs px, mobile-first approach.
- **CSS animations:** @keyframes, transition property, transform for GPU-accelerated smooth motion.
- **Bootstrap:** 12-column grid system, utility/helper classes.

## 9. SQL & Databases

---

*Reported directly: candidates were asked to write SQL queries live on a notepad — practice writing syntactically correct SQL by hand, not just in an IDE with autocomplete.*

- **Joins:** INNER (matching rows only), LEFT (all left + matches), RIGHT, FULL OUTER (all rows both sides) — be able to describe each with a 2-table example.
- **GROUP BY + HAVING** (filters aggregated groups) **vs WHERE** (filters rows before aggregation).
- **Normalization (1NF-3NF):** reduces redundancy/anomalies; denormalization trades that back for read performance.
- **Indexes:** speed up reads via faster lookup structures (commonly B-trees), but slow down writes and use extra storage; clustered (physically orders table data) vs non-clustered.
- **Keys:** Primary key (unique, non-null, one per table) vs Foreign key (references another table's PK) vs Unique key (allows one null in most DBs).
- **ACID:** Atomicity, Consistency, Isolation, Durability.
- **Isolation levels:** Read Uncommitted, Read Committed, Repeatable Read, Serializable — and what dirty reads / phantom reads mean.
- **SQL injection prevention:** parameterized queries / PreparedStatements (ties directly into the JD's security-vulnerabilities bullet).

**Q: Write a query for the second-highest salary in an Employee table.**

SELECT MAX(salary) FROM Employee WHERE salary < (SELECT MAX(salary) FROM Employee); — or use DENSE\_RANK() with a window function for the Nth-highest generalized version.

**Q: Difference between WHERE and HAVING?**

WHERE filters individual rows before grouping; HAVING filters groups after GROUP BY aggregation (e.g., HAVING COUNT(\*) > 5).

**Q: What is an index and why not index every column?**

An index creates a faster lookup path for reads, but every additional index adds overhead to INSERT/UPDATE/DELETE operations and consumes storage — so indexes are added selectively on frequently-queried columns.

## 10. Data Structures & Algorithms

---

*Reported directly as an interview topic (basic DSA, live-coded). Depth expected is moderate for an intern/analyst role — focus on clean explanation and correctness over optimal complexity tricks.*

- **Complexity basics:** know Big-O for common ops on Array, LinkedList, HashMap, Stack, Queue, BST, Heap.
- **Core structures:** Arrays, Linked Lists (singly/doubly), Stacks, Queues, Trees (BST, inorder/preorder/postorder traversal), Graphs (BFS/DFS), HashMaps, Heaps.
- **Sorting:** know Merge Sort and Quick Sort at least — complexity, stability, worst-case behavior.
- **Recursion:** base case + recursive case; be able to trace a recursive function by hand step by step.
- **Flowcharts (explicitly named in JD):** be comfortable translating a simple algorithm (e.g., login validation, order-processing flow) into a flowchart with decision boxes — this may come up as a whiteboard/verbal exercise, not just code.

### Have these ready to code cold, on paper or a notepad

- Reverse a string / array / linked list
- Check for a palindrome
- Two-sum problem
- FizzBuzz
- Balanced parentheses (stack-based)
- Binary search (iterative + recursive)
- Find the missing number in an array of 1..N
- Detect a cycle in a linked list

## 11. System Design / Distributed Systems Basics

---

*Reported directly for tech-focused MUFG interviews: distributed systems, Docker, and Kafka awareness came up alongside multithreading.*

- **3-tier architecture:** Presentation (client/UI) → Application/Business logic (server/API) → Data (DB).
- **Monolith vs Microservices:** monolith = simpler to deploy/debug but harder to scale parts independently; microservices = independent scaling/deployment but adds network/orchestration complexity.
- **Message queues / Kafka (conceptual level):** decouples producers and consumers, enables async event-driven communication and buffering against traffic spikes.
- **Load balancing:** distributes requests across multiple server instances (e.g., round robin) for availability and scale.
- **Caching:** where it fits (client, CDN, server, DB layer) and the basic idea of cache invalidation.
- **Statelessness** in REST services enabling easy horizontal scaling.
- **Containers (Docker):** packages an app + its dependencies into a portable, consistent runtime unit — contrast with a full VM (containers share the host OS kernel, so they're lighter weight).

### **Q: How would you design a simple order-processing system across a UI, API, and database?**

Frame it as: client submits order → API validates & writes to DB in a transaction → (optionally) publishes an event to a queue for downstream processing (e.g., notifications, inventory update) → returns a status to the client. Mention statelessness of the API and where you'd add caching/indexing for scale.

## 12. Git

---

*Reported directly as a topic asked alongside AWS/Terraform/CI-CD in a technical round.*

- Core commands: clone, commit, push, pull, fetch, branch, merge, rebase, stash, cherry-pick.
- **Merge vs Rebase:** merge creates a new merge commit preserving both histories; rebase replays your commits on top of another branch, producing a linear (but rewritten) history — never rebase shared/public branches.
- Resolving a merge conflict: Git marks conflicting sections in the file (<<<<<< / ===== / >>>>>>), you manually edit to the correct final content, then git add + commit (or continue the rebase).
- **git fetch vs git pull:** fetch downloads remote changes without merging into your branch; pull = fetch + merge (or rebase) in one step.
- .gitignore, git log, git diff, HEAD, detached HEAD state.
- Branching strategy basics: feature branches, Git Flow.

## 13. Application Security (OWASP)

---

- **SQL Injection** → mitigate with parameterized queries / PreparedStatements.
- **XSS (Cross-Site Scripting)** → mitigate with output encoding and Content-Security-Policy headers.
- **CSRF (Cross-Site Request Forgery)** → mitigate with anti-CSRF tokens and SameSite cookies.
- **Broken Authentication** → strong session management, MFA, secure password storage (hashing + salting).
- **Sensitive Data Exposure** → encryption at rest and in transit (HTTPS/TLS).
- **Security Misconfiguration & Insecure Deserialization** — know these by name at minimum.
- **CORS:** browser-enforced policy restricting which origins can call your API — why it exists (prevents malicious sites from silently using a logged-in user's session against another site).
- **JWT:** structure is header.payload.signature; used for stateless authentication.

### Q: What is XSS and how do you prevent it in a React app?

XSS is injecting malicious script into a page that then executes in another user's browser. React escapes content rendered via JSX by default, but you must be careful with `dangerouslySetInnerHTML` and always sanitize/encode any raw HTML you do render.

## 14. Agile / Scrum / Waterfall

---

- **Waterfall:** strictly sequential phases (Requirements → Design → Implementation → Verification → Maintenance) — hard to accommodate changing requirements mid-project.
- **Scrum:** time-boxed sprints (commonly 2 weeks); roles = Product Owner, Scrum Master, Dev Team; ceremonies = Sprint Planning, Daily Standup, Sprint Review, Retrospective; artifacts = Product Backlog, Sprint Backlog, Increment.
- **Kanban vs Scrum:** Kanban is continuous flow with WIP limits, no fixed sprint cadence, vs Scrum's fixed time-boxed iterations.
- Know when Agile is preferable to Waterfall: evolving/unclear requirements, need for frequent stakeholder feedback.

## 15. AWS, Docker & CI/CD Basics

---

*Reported directly: a technical round asked 'standard AWS services, Terraform, Git and CI/CD'. Keep this at a solid conceptual level rather than deep hands-on AWS console knowledge.*

- **EC2:** virtual compute instances. **S3:** object storage (vs EBS, which is block storage attached to an EC2 instance). **RDS:** managed relational database service.
- **Lambda:** serverless — runs code in response to events without managing servers; good for short, event-driven tasks.
- **IAM:** identity & access management — principle of least privilege.
- **VPC:** isolated virtual network for your AWS resources.
- **CloudWatch:** monitoring, logging, and alerting.
- **Auto Scaling + Elastic Load Balancer:** automatically adjusts compute capacity and distributes traffic.
- **Terraform (Infrastructure as Code):** declaratively defines cloud infrastructure in config files so environments are reproducible and version-controlled, instead of manual console clicking.
- **Docker:** packages an application with its dependencies into a portable container image; 'it works the same in dev, test, and prod.'
- **CI/CD basics:** Continuous Integration (automatically build/test on every commit) and Continuous Deployment/Delivery (automatically ship validated changes) — name a tool (Jenkins/GitHub Actions/GitLab CI) if you've used one.

## 16. LLMs / GenAI Basics

---

*Explicitly listed in the JD's secondary skills — don't skip this even if it feels tangential.*

- **What an LLM is:** a transformer-based model trained to predict the next token; behavior is shaped by pretraining plus fine-tuning/prompting, not hard-coded rules.
- **Prompt engineering:** zero-shot vs few-shot prompting; system vs user prompts; asking for structured (e.g., JSON) output reliably.
- **RAG (Retrieval-Augmented Generation):** grounds model responses in retrieved external data to reduce hallucination and add up-to-date/private knowledge.
- **AI Agents:** a system where an LLM plans, calls tools/APIs, and acts iteratively toward a goal — distinct from a single-shot prompt/response chatbot call.
- **Agent orchestration:** one agent delegating subtasks to other specialized agents, or a pipeline of agents each handling a stage.
- **Model families to recognize:** GPT (OpenAI), Claude (Anthropic), Gemini (Google), plus open-source options like Llama/Mistral.
- **Practical integration:** calling an LLM API from a backend (e.g., Spring Boot) endpoint, parsing structured JSON responses, handling API errors/timeouts gracefully.

### **Q: What is the difference between prompting and fine-tuning?**

Prompting shapes model behavior at inference time via instructions/examples with no change to model weights; fine-tuning further trains the model's weights on a specific dataset, which is more durable but expensive and shifts the model itself.

### **Q: What makes something an 'agent' rather than a simple chatbot call?**

An agent doesn't just answer once — it can reason about a goal, decide to call external tools/APIs, observe the result, and iterate through multiple steps autonomously until the goal is met.

## 17. HR & Behavioral Round

---

*This is weighted very heavily at MUFG across nearly every reported interview — often more than the technical rounds. Prepare tight, specific answers, not generic ones.*

### Confirmed/likely questions (directly reported by MUFG candidates)

- Tell me about yourself.
- What are your strengths and weaknesses?
- Walk me through your resume.
- Why MUFG? Why banking/financial services?
- Tell me about a challenge you faced in a project, and how you handled it.
- Situational/behavioral prompts: 'Describe a time you disagreed with a teammate', 'Describe a time you had to learn something quickly', 'How do you handle tight deadlines?'
- What do you expect from management? (reported specifically)
- General current-affairs/industry-awareness questions have come up in some rounds — be ready to speak briefly about a recent development in banking/fintech or AI if asked.

### Prep approach

- Use the STAR format (Situation, Task, Action, Result) for every behavioral answer — keep each to 60-90 seconds.
- Have 2-3 concrete technical project stories ready (a bug you debugged, a feature you designed, a tradeoff you made) that you can adapt to whichever behavioral question is asked.
- For 'Why MUFG' — mention it's Japan's largest bank with a genuinely global footprint, and connect your own interest in fintech/software engineering to the kind of large-scale systems work a bank's tech org does. Keep this short and sincere, not a memorized script.
- Candidates consistently describe the tone as conversational and relaxed rather than adversarial — calm, clear, concise answers land better than over-rehearsed ones.

## 18. Test-Day Strategy & Priority Checklist

---

### If your remaining prep time is short, prioritize in this order:

- **1. JavaScript fundamentals** — closures, hoisting, this, async/event loop. Heavily tested via 'what does this print' style questions.
- **2. React hooks + Redux data flow** — be ready to explain the 'why', not just recite hook names.
- **3. Core Java OOP + Collections + HashMap internals** — plus one live-coded OOP example (abstraction/overriding) since this was directly reported.
- **4. SQL joins + one query written by hand** (e.g., second-highest salary, GROUP BY + HAVING) — reported as literally written on a notepad in interviews.
- **5. OWASP top 3** (SQLi, XSS, CSRF) + **Git merge vs rebase**.
- **6. Quant + logical reasoning refresh** — 20-30 min of timed practice questions to get your speed back, especially percentages, time-work, and syllogisms.
- **7. Agile ceremonies + AWS/Docker/CI-CD vocabulary** — quick recall-level review, not deep study.
- **8. LLM/GenAI vocabulary** — enough to speak fluently for 60-90 seconds if it comes up.
- **9. Lock in 2-3 STAR behavioral stories** — this is reported as the most consistently asked category across every round type.

### Day-of reminders

- Read every question fully before answering — MUFG's process is described as conversational; talking through your reasoning out loud matters as much as the final answer.
- For live coding, narrate your thought process — interviewers have explicitly praised candidates who explain approach before/while writing code.
- If you don't know something, say so and reason toward an approximate answer rather than guessing silently — several reviews note interviewers value transparency and communication over perfect recall.
- Keep behavioral answers tight (60-90 sec) — rambling was noted as a negative in multiple reviews.